Fejlesztő adatai:

- Név: Ballai Péter Lóránt
- Neptun kód: EYSEED

1. Feladatleírás

Készítsünk programot, amellyel a következő két személyes játékot lehet játszani. Adott egy n*n mezőből álló tábla, amelyen kezdetben a játékosoknak n fehér, illetve n fekete kavics áll rendelkezésére, amelyek elhelyezkedése véletlenszerű. A játékosok kiválaszthatnak egy saját kavicsot, amelyet függőlegesen vagy vízszintesen eltolhatnak. Eltoláskor azonban nem csak az adott kavics, hanem a vele az eltolás irányában szomszédos kavicsok is eltolódnak, a szélső mezőn lévők pedig lekerülnek a játéktábláról. A játék célja, hogy adott körszámon belül (5n) az ellenfél minél több kavicsát letoljuk a pályáról (azaz nekünk maradjon több kavicsunk a végére). Ha mindkét játékosnak ugyanannyi marad, akkor a játék döntetlen. A program biztosítson lehetőséget új játék kezdésére a pályaméret (3*3, 4*4, 6*6) és így a lépésszám (15, 20, 30) megadásával, ismerje fel, ha vége a játéknak (jelenítse meg a győztest), majd automatikusan kezdjen új játékot. Ezen felül legyen lehetőség a játék elmentésére, valamint betöltésére.

2. **UML Felhasználói Esetek (Use Case)**

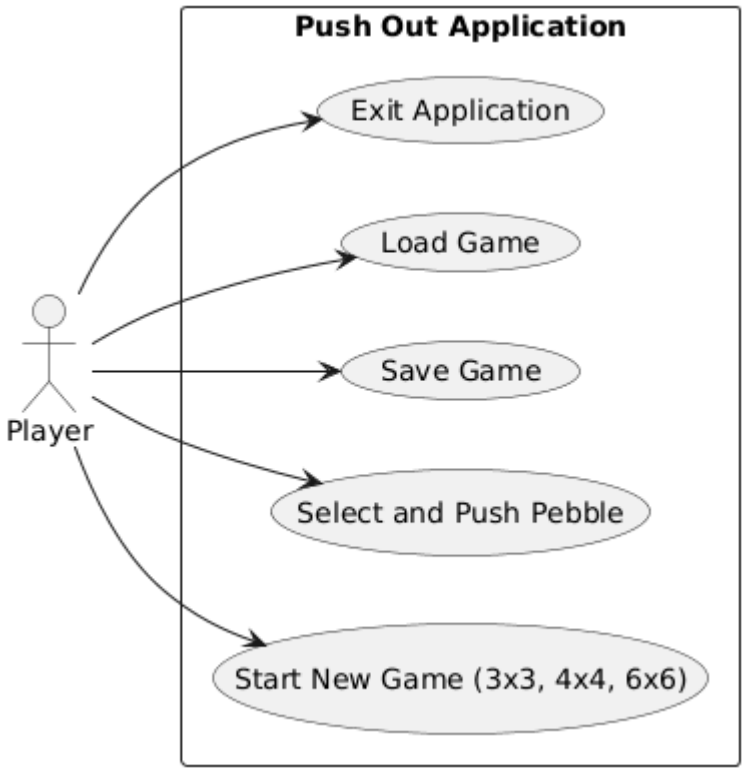
A feladat elemzése: A játékot egy 3*3, 4*4 vagy 6*6 méretű négyzetrácsos táblán játssza két játékos (Fehér és Fekete). A program a játék kezdetén véletlenszerűen elhelyez a táblán n darab fehér és n darab fekete kavicsot, míg a többi mező üres marad. A játék során a játékosok felváltva lépnek, és rendelkezésére áll egy körszámláló, amely kezdetben 5n lépést enged meg.

Egy lépés során a soron következő játékos kijelöli az egyik saját kavicsát, majd egy szomszédos üres vagy másik kaviccsal foglalt mező felé (vagy a kijelölést egy szomszédos saját kavicsra kattintva irányt adva) eltolja azt. A láncolatos eltolás elve alapján a kiválasztott kavics és az eltolás irányában előtte lévő összes többi kavics elmozdul egy mezővel. Ha a lánc végén lévő kavics a tábla szélén túlra csúszna, az leesik a pályáról és végleg eltűnik. Minden érvényes eltolás után csökken a hátralévő lépések száma, és a kör átkerül a másik játékoshoz. A játék akkor ér véget, ha a lépések száma eléri a nullát, vagy valamelyik játékos összes kavicsa leesik a pályáról. Ekkor a program megjeleníti a győztest (vagy döntetlent), majd automatikusan új játékot indít.

Felhasználói esetek:

- **Új játék indítása:** A felhasználó a menüből kiválasztja a kívánt pályaméretet, mire a program generál egy új, véletlenszerű kavicsegyensúlyú táblát és beállítja a korlátozott lépésszámot.

- **Kavics kiválasztása és eltolása:** A játékos a saját színű kavicsára kattint, majd megadja az eltolás irányát (egérrel vagy nyílbillentyűkkel), mire a program végrehajtja a láncolatos tolét és frissíti a lépésszámlálót.
- **Játék mentése:** A játékos a menüből elmentheti az aktuális táblaállapotot, a lépésszámot és a soron lévő játékost egy fájlba.
- **Játék betöltése:** A felhasználó egy korábban mentett fájlból visszatöltheti a játékállást.
- **Kilépés:** A felhasználó bezárja az alkalmazást.



3. **UML Osztálydiagram (Class Diagram)**
Az alkalmazás háromrétegű architektúrában valósul meg (PushOut nézet, PushOut.Model üzleti logika és perzisztencia).

Perzisztencia réteg

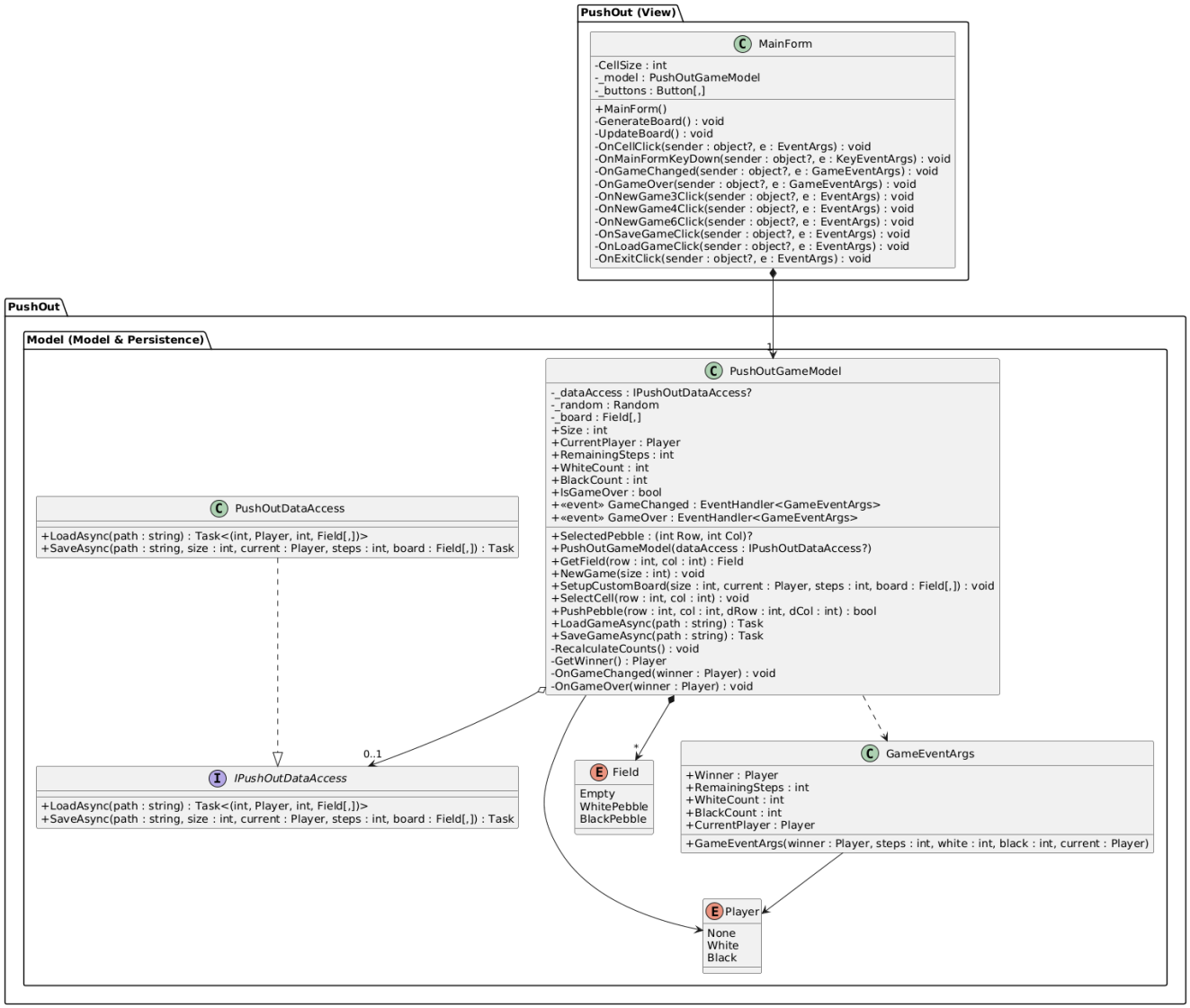
- **IPushOutDataAccess (interfész):** Definiálja a mentési (SaveAsync) és betöltési (LoadAsync) aszinkron metódusokat. Biztosítja a modell függetlenségét a fájlkezeléstől, és lehetővé tétele a tesztek során a mockolást.
- **PushOutDataAccess (osztály):** Megvalósítja az interfészt szöveges állománykezeléssel (StreamReader, StreamWriter). A fájl első sora a táblaméretet, az aktuális játékost és a hátralévő lépésszámot tárolja, míg a többi sor a tábla mátrixának celláit.

Modell réteg (PushOut.Model)

- **Player (enum):** A játékosok megjelölésére szolgál (None, White, Black).
- **Field (enum):** A mezők tartalmát írja le (Empty, WhitePebble, BlackPebble).
- **GameEventArgs (osztály):** Az eseménykezeléshez használt adatosztály, amely tárolja a győztest, a hátralévő lépéseket, a két játékos fennmaradó kavicsainak számát, valamint az aktuális játékost.
- **PushOutGameModel (osztály):** A központi logikai osztály.
- Nyilvántartja a táblát (_board), a méretet (Size), a körön lévő játékost (CurrentPlayer), a hátralévő lépéseket (RemainingSteps) és a kavicsok darabszámait (WhiteCount, BlackCount).
- A NewGame(int size) metódus létrehozza a táblát, és véletlenszerűen elhelyezi rajta a fehér és fekete kavicsokat.
- A PushPebble metódus kezeli a láncolatos eltolás logikáját, ellenőrzi a pályahatárokat, kiszedi a lelőtt kavicsokat, csökkenti a lépésszámlálót, és kiszámítja a győzelmi kondíciókat.
- Eseményeket (GameChanged, GameOver) vált ki az állapotváltozásokról.

Nézet réteg (PushOut)

- **MainForm (osztály):** A Windows Forms felületért felelős ablak, amely tartalmazza a menüsort, az állapotsort és a dinamikusan felépülő gombmátrixot (Button[,]).
- Dinamikusan igazítja az ablak méretét a játéktábla méretéhez, kezeli az egérkattintásokat és a billentyűzetet (nyílbillentyűk/WASD a tolításhoz), valamint feliratkozik a modell eseményeire.


PushOut (View)
MainForm
-CellSize : int
-_model : PushOutGameModel
-_buttons : Button[,]
+MainForm()
-GenerateBoard() : void
-UpdateBoard() : void
-OnCellClick(sender : object?, e : EventArgs) : void
-OnMainFormKeyDown(sender : object?, e : KeyEventArgs) : void
-OnGameChanged(sender : object?, e : GameEventArgs) : void
-OnGameOver(sender : object?, e : GameEventArgs) : void
-OnNewGame3Click(sender : object?, e : EventArgs) : void
-OnNewGame4Click(sender : object?, e : EventArgs) : void
-OnNewGame6Click(sender : object?, e : EventArgs) : void
-OnSaveGameClick(sender : object?, e : EventArgs) : void
-OnLoadGameClick(sender : object?, e : EventArgs) : void
-OnExitClick(sender : object?, e : EventArgs) : void
1
PushOut
Model (Model & Persistence)
PushOutGameModel
-_dataAccess : IPushOutDataAccess?
-_random : Random
-_board : Field[,]
+Size : int
+CurrentPlayer : Player
+RemainingSteps : int
+WhiteCount : int
+BlackCount : int
+IsGameOver : bool
+«event» GameChanged : EventHandler<GameEventArgs>
+«event» GameOver : EventHandler<GameEventArgs>
+SelectedPebble : (int Row, int Col)?
+PushOutGameModel(dataAccess : IPushOutDataAccess?)
+GetField(row : int, col : int) : Field
+NewGame(size : int) : void
+SetupCustomBoard(size : int, current : Player, steps : int, board : Field[,]) : void
+SelectCell(row : int, col : int) : void
+PushPebble(row : int, col : int, dRow : int, dCol : int) : bool
+LoadGameAsync(path : string) : Task
+SaveGameAsync(path : string) : Task
-RecalculateCounts() : void
-GetWinner() : Player
-OnGameChanged(winner : Player) : void
-OnGameOver(winner : Player) : void
PushOutDataAccess
+LoadAsync(path : string) : Task<(int, Player, int, Field[,])>
+SaveAsync(path : string, size : int, current : Player, steps : int, board : Field[,]) : Task
IPushOutDataAccess
+LoadAsync(path : string) : Task<(int, Player, int, Field[,])>
+SaveAsync(path : string, size : int, current : Player, steps : int, board : Field[,]) : Task
0..1
Field
Empty
WhitePebble
BlackPebble
*
GameEventArgs
+Winner : Player
+RemainingSteps : int
+WhiteCount : int
+BlackCount : int
+CurrentPlayer : Player
+GameEventArgs(winner : Player, steps : int, white : int, black : int, current : Player)
Player
None
White
Black

4. Tesztesetek

A modell helyes működését MSTest egységtesztek igazolják a MockDataAccess segítségével:

1. **NewGame_InitializesBoardCorrectly:** Ellenőrzi, hogy a különböző méretekhez tartozó kezdőlépésszámok és a kavicsmennyiségek helyesen inicializálódnak-e.
2. **PushPebble_ChainPushAndFallOffEdge_UpdatesBoardAndCounts:** Teszteli a láncolatos tolítást és azt, hogy a tábla széléről letolt kavics helyesen eltűnik-e a pályáról, csökkentve az adott játékos darabszámát.
3. **PushPebble_LastStep_TriggersGameOverWithCorrectWinner:** Ellenőrzi a lépésszámláló lejártakor bekövetkező játékvége állapotot és a győztes helyes megállapítását a fennmaradó kavicsok alapján.
4. **PushPebble_DrawCondition_FiresGameOverWithNone:** Teszteli a döntetlen szituációt, amikor egyenlő számú kavics marad a táblán a lépések elfogytakor.
5. **PushPebble_InvalidMove_ReturnsFalse:** Biztosítja, hogy üres mező eltolási kísérlete vagy ellenfél kavicsának mozgatása érvénytelen maradjon, és ne módosítsa a játékállást.
6. **SaveAndLoadGame_UsesDataAccessCorrectly:** Ellenőrzi a perzisztencia működését a mentési és betöltési adatok helyes átadásán keresztül.